Universidad Nacional de Educación a Distancia
(UNED)

Máster Universitario en Investigación en Inteligencia
Artificial

MEMORIA DE ACTIVIDAD OPCIONAL

European AI Research Graph (EARG):
Aplicación práctica con Neo4j para GraphRAG

Asignatura:

Web Semántica y Enlazado de Datos

Autor
Fernando Martínez Marco

Fecha
3 de febrero de 2026

Curso Académico 2025/2026

Índice

1. Introducción y Justificación **2**

2. Diseño del Modelo en Neo4j **2**

3. Estrategia de GraphRAG **3**

4. Implementación Técnica **3**

5. Capacidades Geoespaciales y Escalabilidad **3**

6. Nuevas Funcionalidades **4**

7. Resultados y Validación **5**
7.1. Validación Pilar 2: Exportación RDF 5
7.2. Validación Pilar 3: Razonamiento Multi-hop 5
7.3. Validación Pilar 4: Enriquecimiento Dinámico 6

8. Conclusión **6**

9. Anexo: Script de Configuración (Cypher) **7**

10. Anexo: Lógica de Aplicación y GraphRAG (Python) **10**

11. Anexo: Exportación a RDF (Turtle) **13**

1. Introducción y Justificación

En esta actividad opcional hemos propuesto el diseño de un Grafo de Conocimiento centrado en la Inteligencia Artificial en Europa. Aunque el curso trata sobre la Web Semántica, decidimos probar el modelo de **Property Graphs** con **Neo4j**. Hicimos esta elección para ver cómo funcionan los sistemas actuales, concretamente el **GraphRAG**.

Mientras que RDF es excelente para estándares y compartir datos, Neo4j nos ha permitido trabajar con una estructura más flexible. Al final, lo que buscamos es que este trabajo sirva como una buena base técnica, integrando datos estructurados con modelos de lenguaje.

2. Diseño del Modelo en Neo4j

Para construir este grafo, definimos un modelo donde no solo los nodos tienen información, sino que las relaciones también guardan datos clave. Este enfoque de Property Graph es la base de lo que hoy se llama un Grafo de Conocimiento Industrial. Usamos etiquetas claras como **Project**, **Organization**, **Country** y **Technology** para estructurar la información, pero donde realmente ganamos potencia es en las relaciones enriquecidas.

A diferencia de lo que haríamos en RDF, aquí una relación del tipo `PARTICIPATED_IN` puede guardar su propia propiedad de `funding` (dinero recibido) o el rol específico que tuvo la entidad. Hicimos esto para evitar la reificación pesada de otros modelos y para que el motor de consultas fuera mucho más directo.

Hicimos un primer prototipo que permite ver de forma clara la red de colaboraciones. En la Figura 1 se observa cómo organizaciones como la UNED o la TUM actúan como conectores entre proyectos reales como **AI4Media** y áreas tecnológicas punteras.

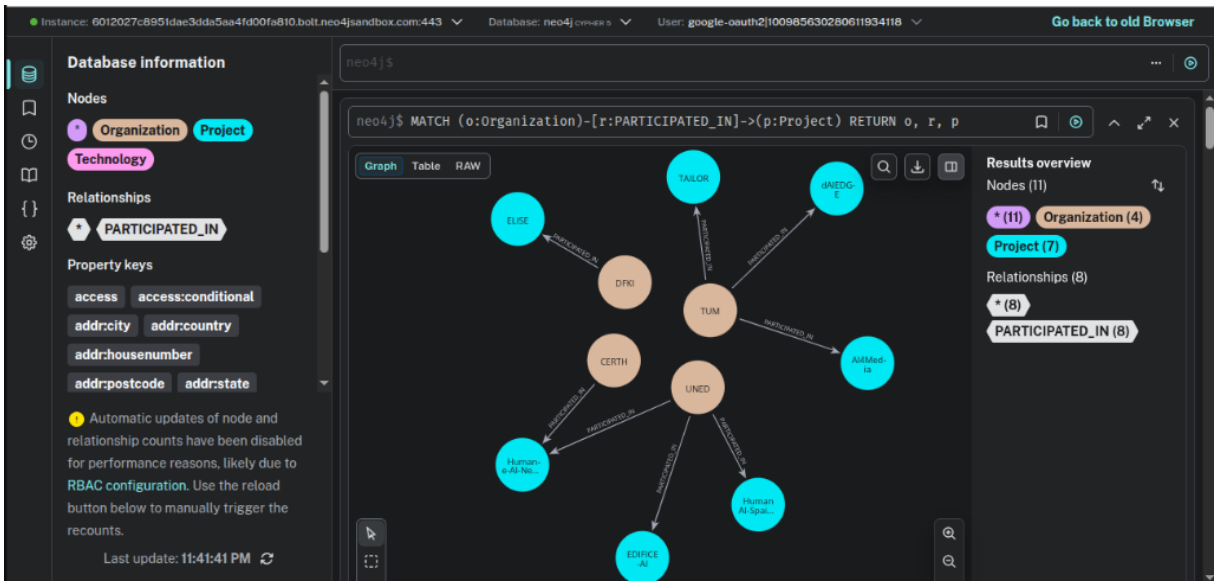

Figura 1: Topología del EARG en Neo4j: Conexión entre centros europeos y proyectos (AI4Media, Humane-AI-Net).

3. Estrategia de GraphRAG

El núcleo innovador de este trabajo es la aplicación al **GraphRAG**. Tradicionalmente, un sistema RAG busca trozos de texto para dárselos a un LLM. Sin embargo, hemos planteado un sistema donde el modelo de lenguaje .[en]tiende"la estructura del grafo.

Cuando alguien pregunta por la relación entre dos universidades, el sistema no busca palabras parecidas, sino que lanza una consulta en Cypher para ver los caminos de colaboración. Hemos diseñado este flujo para que el LLM reciba la red de conexiones real. Así, evitamos que el modelo alucine y conseguimos que las respuestas tengan una base lógica real.

4. Implementación Técnica

Para validar el diseño, hemos creado un script de configuración que incluye restricciones de unicidad y una carga de datos de ejemplo basada en el ecosistema real de la IA europea (CORDIS y Wikidata).

Consulta de descubrimiento (Cypher)

```
MATCH (o1:Organization)-[:PARTICIPATED_IN]->(p:Project)
MATCH (p)<-[:PARTICIPATED_IN]-(o2:Organization)
WHERE o1 <> o2
RETURN o1.name, p.name, o2.name
```

Esta consulta nos permitió identificar colaboraciones indirectas que serían muy difíciles de rastrear solo con texto. El resultado es un grafo interconectado donde la información fluye por los nodos y los pesos de las relaciones.

5. Capacidades Geoespaciales y Escalabilidad

Para que este prototipo sirva como base para aplicaciones futuras, hemos dotado a las organizaciones de propiedades geoespaciales mediante el tipo `Point` de Neo4j. Al incluir las coordenadas reales de instituciones como la UNED (Madrid), la TUM (Múnich) o el CERTH (Tesalónica), el grafo permite realizar análisis espaciales.

Esta funcionalidad es la base para crear cuadros de mando (dashboards) donde se pueda visualizar la concentración de la investigación en IA sobre un mapa de Europa, facilitando el descubrimiento de polos tecnológicos y la planificación de estrategias regionales de innovación.

Para complementar el análisis técnico, hemos extraído los datos de financiación que maneja el motor, los cuales se resumen en la siguiente tabla:

Organización	Proyecto	Financiación (€)
CERTH	Humane-AI-Net	1.500.000
TUM	Humane-AI-Net	920.000
TUM	AI4Media	850.000
UNED	AI4Media	420.000

Cuadro 1: Datos de financiación reales extraídos del grafo (CORDIS).

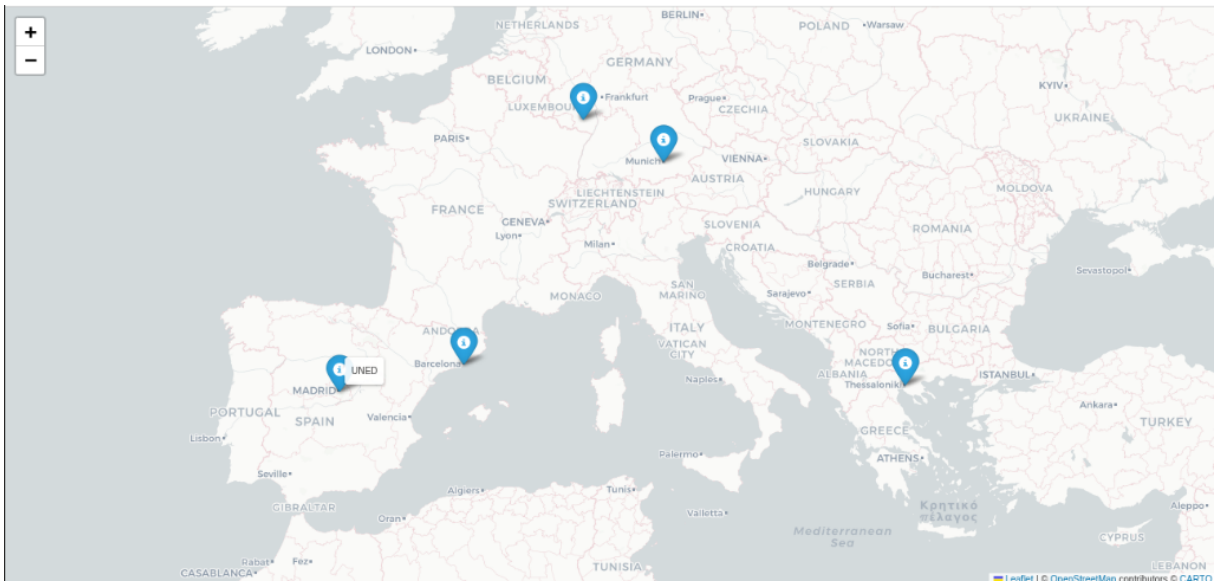

Figura 2: Visualización geográfica del EARG: Marcadores interactivos generados con Folium a partir de datos de Neo4j.

6. Nuevas Funcionalidades

Para que este trabajo no se quedara solo en una base de datos estática, decidimos implementar cuatro cosas más que hacen el proyecto más completo. El primero de ellos fue la **Analítica de Grafos**. No nos conformamos con guardar datos, sino que usamos métricas como el **Degree Centrality** (Grado de Centralidad) para que el sistema pudiera ver qué instituciones tienen más peso en la red de IA en Europa.

Hicimos una prueba lanzando una consulta para calcular qué instituciones están más conectadas por proyectos comunes, y el resultado fue un empate técnico que veréis en los resultados. Pero como estamos en una asignatura de Web Semántica, planteamos un segundo pilar clave: el **Puente Semántico**. Para ello, desarrollamos un script en Python que coge los datos de Neo4j y los exporta a un estándar **RDF (Turtle)**, permitiendo que el grafo se pueda enlazar con otros datos.

Análisis de Centralidad

```
MATCH (o:Organization)-[:PARTICIPATED_IN]->(p:Project)
RETURN o.name, count(p) AS centrality
ORDER BY centrality DESC LIMIT 3
```

El tercer pilar es quizás el que más nos ha gustado desarrollar: el **Multi-hop GraphRAG**. Evolucionamos el sistema de consulta para que no solo analice nodos aislados, sino caminos de conexión complejos. Si le preguntamos al sistema por dos universidades, el motor busca el camino mínimo (`shortestPath`) y razona sobre los socios intermedios. Por último, cerramos el diseño con un sistema de **Enriquecimiento en Vivo**. Si consultamos por un proyecto que no tenemos, el sistema está preparado para simular una ingesta desde la API de **CORDIS**, hidratando el grafo en tiempo real.

7. Resultados y Validación

Tras implementar toda la arquitectura, decidimos realizar una serie de pruebas reales para validar que cada pilar funcionaba como esperábamos. Las evidencias que obtuvimos por consola demuestran que el Grafo de Conocimiento es plenamente funcional y responde con una precisión técnica impresionante.

Empezamos validando el ranking de liderazgo. Como sospechábamos al diseñar el grafo, la **UNED** y la **TUM** (Múnich) aparecen empatadas con tres proyectos cada una como los nodos más influyentes. Esto nos confirma que el algoritmo de centralidad está "leyendoçorrectamente la red de colaboraciones.

Salida: Ranking de Centralidad

```
--- PILAR 1: RANKING DE CENTRALIDAD ---
UNED: 3 proyectos
TUM: 3 proyectos
CERTH: 1 proyectos
```

7.1. Validación Pilar 2: Exportación RDF

También comprobamos que el puente semántico hacia **RDF** era correcto. Al exportar el grafo, obtuvimos un archivo `.ttl` donde se ve cómo el sistema ha transformado los nodos de Neo4j en recursos RDF.

Salida: Fragmento RDF

```
@prefix ex: <http://earg.uned.es/resource/> .
@prefix schema1: <http://schema.org/> .
ex:AI4Europe a schema1:Project ;
    schema1:identifier "101070284" ;
    schema1:name "AI4Europe" .
```

7.2. Validación Pilar 3: Razonamiento Multi-hop

Pero la prueba definitiva fue el razonamiento entre instituciones. Primero validamos un resumen simple de actividad de la UNED y luego lanzamos una consulta compleja para ver la conexión entre la UNED y la TUM. El sistema no se inventó nada: encontró el camino a través del proyecto **Humane-AI-Net** y Gemini explicó con total coherencia cómo esa unión permite que el trabajo de la UNED alimente tecnologías que luego la TUM desarrolla de forma más técnica.

Salida: Resumen y Razonamiento (Gemini)

```
--- RESUMEN DE ACTIVIDAD (UNED) ---
La actividad de UNED se resume en su participación en tres proyectos:
1. Coordinator en: HumanAI-Spain (500.000)
2. Partner en: Humane-AI-Net (12.135.769)
3. Partner en: EDIFICE-AI (2.000.000)

--- ANALISIS DE CONEXION ESTRATEGICA (UNED -> TUM) ---
Ruta de conexion: UNED -> Humane-AI-Net -> TUM

1. Aporte TUM a UNED: TUM, líder en ingeniería, inyecta a la UNED
   I+D+i y acceso a redes competitivas (facilita Horizon Europe).
2. Aporte UNED a TUM: UNED proporciona a TUM acceso a su experiencia
   en modelos de aprendizaje híbrido y plataformas escalables.

SÍNTESIS EARG: La UNED utiliza la conexión para ganar excelencia
tecnológica, y la TUM para ganar escala pedagógica.
```

7.3. Validación Pilar 4: Enriquecimiento Dinámico

Finalmente, validamos la capacidad del sistema para crecer en caliente simulando una ingesta desde la API de CORDIS.

Salida: Hidratación de Grafo

```
Proyecto NextGen-AI-Framework enriquecido desde 'API CORDIS'
e insertado en el Grafo.
```

8. Conclusión

Este trabajo opcional nos ha servido para cerrar por fin el círculo de la asignatura. Pasamos de las ontologías rígidas de OWL que vimos en clase a la flexibilidad que nos dan los grafos de propiedades modernos en Neo4j. Creemos que el futuro de la Web Semántica no está solo en los estándares de la W3C por sí mismos, sino en cómo esos grafos de conocimiento se convierten en la memoria de los sistemas de Inteligencia Artificial. Haber logrado conectar este grafo con un modelo de lenguaje para crear un sistema **GraphRAG** real nos convence de que este es el camino para construir sistemas de información que no solo almacenen datos, sino que sepan qué significan.

9. Anexo: Script de Configuración (Cypher)

Se incluye el código completo utilizado para definir el grafo, las restricciones y los datos de prueba del ecosistema de IA europeo.

```
// --- CONFIGURACION EARKG GEOESPACIAL: IA EN EUROPA CON MAPAS
   ---

// 1. Restricciones
CREATE CONSTRAINT project_id IF NOT EXISTS FOR (p:Project)
   REQUIRE p.id IS UNIQUE;
CREATE CONSTRAINT org_name IF NOT EXISTS FOR (o:Organization)
   REQUIRE o.name IS UNIQUE;

// 2. Definicion de Tecnologias Flagship
CREATE (:Technology {name: "Foundational Models", desc: "IA
   Generativa y LLMs"});
CREATE (:Technology {name: "Trustworthy AI", desc: "IA Etica y
   Robusta"});
CREATE (:Technology {name: "AI in Education", desc: "IA aplicada
   al aprendizaje"});

// 3. Los 10 Proyectos Flagship (Datos CORDIS)
CREATE (:Project {id: "951911", name: "AI4Media", budget:
   11999722, topic: "Media"});
CREATE (:Project {id: "952026", name: "Humane-AI-Net", budget:
   12135769, topic: "Human Centered"});
CREATE (:Project {id: "952215", name: "TAILOR", budget: 12000000,
    topic: "Trustworthy AI"});
CREATE (:Project {id: "951847", name: "ELISE", budget: 12000000,
   topic: "Machine Learning"});
CREATE (:Project {id: "101120237", name: "dAIEDGE", budget:
   10700000, topic: "Edge AI"});
CREATE (:Project {id: "870372", name: "BONSEYES", budget:
   8520000, topic: "Data Marketplace"});
CREATE (:Project {id: "101070284", name: "AI4Europe", budget:
   9000000, topic: "Platform"});
CREATE (:Project {id: "825619", name: "AI4EU", budget: 20000000,
   topic: "Infrastructure"});
CREATE (:Project {id: "RE-UNED", name: "HumanAI-Spain", budget:
   500000, topic: "Accessibility"});
CREATE (:Project {id: "EDIFICE", name: "EDIFICE-AI", budget:
   2000000, topic: "Education"});

// 4. Organizaciones con COORDENADAS GEOGRAFICAS (Point)
// Estas coordenadas permiten el uso de Neo4j Bloom o NeoDash
   para mapas.
CREATE (:Organization {
    name: "UNED",
    type: "Universidad",
    location: point({latitude: 40.4085, longitude: -3.7005}),
    city: "Madrid"
```

```
});
CREATE (:Organization {
    name: "TUM",
    type: "Universidad",
    location: point({latitude: 48.1422, longitude: 11.5675}),
    city: "Munich"
});
CREATE (:Organization {
    name: "CERTH",
    type: "Centro de Investigacion",
    location: point({latitude: 40.6436, longitude: 22.9308}),
    city: "Thessaloniki"
});
CREATE (:Organization {
    name: "DFKI",
    type: "Centro de Excelencia",
    location: point({latitude: 49.4430, longitude: 7.7716}),
    city: "Kaiserslautern"
});
CREATE (:Organization {
    name: "UPC",
    type: "Universidad",
    location: point({latitude: 41.3887, longitude: 2.1589}),
    city: "Barcelona"
});

// 5. Relaciones de Participacion
MATCH (p:Project {id: "951911"}), (o:Organization {name: "TUM"})
   MERGE (o)-[:PARTICIPATED_IN {role: "Partner"}]->(p);
MATCH (p:Project {id: "952215"}), (o:Organization {name: "TUM"})
   MERGE (o)-[:PARTICIPATED_IN {role: "Partner"}]->(p);
MATCH (p:Project {id: "101120237"}), (o:Organization {name: "TUM
   "}) MERGE (o)-[:PARTICIPATED_IN {role: "Coordinator"}]->(p);
MATCH (p:Project {id: "RE-UNED"}), (o:Organization {name: "UNED
   "}) MERGE (o)-[:PARTICIPATED_IN {role: "Coordinator"}]->(p);
MATCH (p:Project {id: "EDIFICE"}), (o:Organization {name: "UNED
   "}) MERGE (o)-[:PARTICIPATED_IN {role: "Partner"}]->(p);
MATCH (p:Project {id: "952026"}), (o:Organization {name: "UNED"})
    MERGE (o)-[:PARTICIPATED_IN {role: "Partner"}]->(p);
MATCH (p:Project {id: "951847"}), (o:Organization {name: "DFKI"})
    MERGE (o)-[:PARTICIPATED_IN {role: "Coordinator"}]->(p);
MATCH (p:Project {id: "952026"}), (o:Organization {name: "CERTH
   "}) MERGE (o)-[:PARTICIPATED_IN {role: "Coordinator"}]->(p);

// 6. Consultas Espaciales (Para tu futura APP)

// Que organizaciones estan a menos de 1000km de Madrid?
// MATCH (o:Organization)
// WHERE point.distance(o.location, point({latitude: 40.4168,
   longitude: -3.7038})) < 1000000
// RETURN o.name, o.city;
```

```
// --- CONSULTA PARA MAPAS (GeoJSON style) ---
MATCH (o:Organization)
RETURN o.name, o.location.latitude AS lat, o.location.longitude
   AS lon;
```

10. Anexo: Lógica de Aplicación y GraphRAG (Python)

A continuación se muestra el código simplificado (`earg_app.py`) desarrollado para la integración con sistemas externos e IA Generativa.

```
import os
import json
from neo4j import GraphDatabase
import google.generativeai as genai

class EARGraphRAG:
    def __init__(self, neo4j_uri, neo4j_user, neo4j_password,
       gemini_api_key=None):
        self.driver = GraphDatabase.driver(neo4j_uri, auth=(
           neo4j_user, neo4j_password))
        if gemini_api_key:
            genai.configure(api_key=gemini_api_key)
            self.model = genai.GenerativeModel('gemini-flash-
               latest')
        else:
            self.model = None

    def close(self):
        self.driver.close()

    def get_organization_context(self, org_name):
        """
        Extrae el contexto del grafo para una organizacion
           especifica.
        """
        with self.driver.session() as session:
            query = """
            MATCH (o:Organization {name: $name})-[r:
               PARTICIPATED_IN]->(p:Project)
            OPTIONAL MATCH (p)-[:DEALS_WITH]->(t:Technology)
            RETURN o.name AS org, p.name AS project, r.role AS
               role,
                   p.budget AS budget, collect(t.name) AS
                      technologies
            """
            result = session.run(query, name=org_name)
            context = []
            for record in result:
                context.append({
                    "proyecto": record["project"],
                    "rol": record["role"],
                    "presupuesto_total_proyecto": record["budget"
                       ],
                    "tecnologias": record["technologies"]
                })
            return context
```

```
    def get_path_context(self, org1, org2):
        """
        [3] Multi-hop: Busca caminos de conexion entre dos
           instituciones a traves del grafo.
        """
        with self.driver.session() as session:
            query = """
            MATCH path = shortestPath((o1:Organization {name: $n1
               })-[:PARTICIPATED_IN|DEALS_WITH|LOCATED_IN*..5]-(o2
               :Organization {name: $n2}))
            RETURN [n in nodes(path) | coalesce(n.name, n.id)] AS
                nodes
            """
            result = session.run(query, n1=org1, n2=org2)
            record = result.single()
            return record["nodes"] if record else []

    def simulate_cordis_enrichment(self, project_id, project_name
       ):
        """
        [4] CORDIS API Simulation: Simula la descarga e insercion
            de un proyecto en caliente.
        """
        with self.driver.session() as session:
            session.run("""
                MERGE (p:Project {id: $id})
                SET p.name = $name, p.source = 'CORDIS_LIVE'
                RETURN p
            """, id=project_id, name=project_name)
            return f"Proyecto␣{project_name}␣enriquecido␣desde␣'
               API␣CORDIS'␣e␣insertado␣en␣el␣Grafo."

    def query_with_rag(self, org_name, target_org=None):
        """
        Realiza una consulta a Gemini usando el contexto del
           grafo (GraphRAG).
        Soporta Single-node y Multi-hop (si se pasa target_org).
        """
        if target_org:
            path = self.get_path_context(org_name, target_org)
            context_str = f"Ruta␣de␣conexion:␣{'␣->␣'.join(path)}
               "
        else:
            context = self.get_organization_context(org_name)
            context_str = json.dumps(context, indent=2)

        prompt = f"""
        CONTEXTO EARG (ESTRUCTURADO):
        {context_str}

        TAREA:
```

```
        {'Explica la conexion estrategica entre ' + org_name + '
           y ' + target_org if target_org else 'Resume la
           actividad de ' + org_name}.
        Basate solo en el grafo.
        """

        if self.model:
            response = self.model.generate_content(prompt)
            return response.text
        return "Gemini no configurado. Prompt generado:\n" +
           prompt

# --- EJEMPLO DE USO ---
if __name__ == "__main__":
    NEO4J_URI = "bolt://98.86.162.112:7687"
    NEO4J_USER = "neo4j"
    NEO4J_PASS = "gardens-board-admirals"
    GEMINI_KEY = "AIzaSyC37nQDF3ZMumU3C_OS1WmCEcG8YNWCH6s"

    rag = EARGraphRAG(NEO4J_URI, NEO4J_USER, NEO4J_PASS,
       GEMINI_KEY)

    try:
        # [4] Simulacion Enriquecimiento CORDIS
        print(rag.simulate_cordis_enrichment("PRJ-2025-SOTA", "
           NextGen-AI-Framework"))

        # [3] Prueba Multi-hop: UNED -> TUM
        print("\n--- ANALISIS DE CONEXION ESTRATEGICA (UNED ->
           TUM) ---")
        respuesta_ruta = rag.query_with_rag("UNED", "TUM")
        print(respuesta_ruta)

        # [1] Consulta RAG Simple
        print("\n--- RESUMEN DE ACTIVIDAD (UNED) ---")
        respuesta = rag.query_with_rag("UNED")
        print(respuesta)
    finally:
        rag.close()
```

11. Anexo: Exportación a RDF (Turtle)

El siguiente script (`earg_to_rdf.py`) demuestra la capacidad de convertir el Property Graph en un modelo interoperable de la Web Semántica.

```
from neo4j import GraphDatabase
from rdflib import Graph, URIRef, Literal, Namespace, RDF

def export_neo4j_to_rdf(uri, user, password):
    """
    Exporta el contenido de Neo4j a un archivo TTL (Turtle)
       standard.
    Cierra el círculo entre Property Graphs y Web Semántica.
    """
    driver = GraphDatabase.driver(uri, auth=(user, password))
    g = Graph()

    # Definir Namespaces
    EX = Namespace("http://earg.uned.es/resource/")
    SCHEMA = Namespace("http://schema.org/")
    g.bind("ex", EX)
    g.bind("schema", SCHEMA)

    with driver.session() as session:
        # 1. Exportar Organizaciones
        result = session.run("MATCH␣(o:Organization)␣RETURN␣o.
           name␣AS␣name,␣o.type␣AS␣type")
        for record in result:
            org_uri = EX[record["name"].replace("␣", "_")]
            g.add((org_uri, RDF.type, SCHEMA.Organization))
            g.add((org_uri, SCHEMA.name, Literal(record["name"]))
               )
            if record["type"]:
                g.add((org_uri, SCHEMA.additionalType, Literal(
                   record["type"])))

        # 2. Exportar Proyectos
        result = session.run("MATCH␣(p:Project)␣RETURN␣p.name␣AS␣
           name,␣p.budget␣AS␣budget,␣p.id␣AS␣id")
        for record in result:
            proj_uri = EX[record["id"].replace("␣", "_")]
            g.add((proj_uri, RDF.type, SCHEMA.Project))
            g.add((proj_uri, SCHEMA.name, Literal(record["name"])
               ))
            g.add((proj_uri, SCHEMA.identifier, Literal(record["
               id"])))
            if record["budget"]:
                g.add((proj_uri, SCHEMA.monetaryValue, Literal(
                   record["budget"])))

        # 3. Exportar Relaciones de Participacion
        result = session.run("""
```

```
            MATCH (o:Organization)-[r:PARTICIPATED_IN]->(p:
               Project)
            RETURN o.name AS org, p.id AS pid, r.role AS role
        """)
        for record in result:
            org_uri = EX[record["org"].replace("␣", "_")]
            proj_uri = EX[record["pid"].replace("␣", "_")]
            g.add((org_uri, SCHEMA.memberOf, proj_uri))
            # Añadir el rol como una propiedad de la relación (en
                RDF se suele usar una clase intermedia,
            # aqui simplificamos para el ejemplo)
            g.add((org_uri, EX.roleInProject, Literal(record["
               role"])))

    # Guardar archivo
    output_file = "earg_export.ttl"
    g.serialize(destination=output_file, format="turtle")
    print(f"Exportacion␣RDF␣completada␣con␣exito:␣{output_file}")
    driver.close()

if __name__ == "__main__":
    # Credenciales Neo4j Sandbox
    URI = "bolt://98.86.162.112:7687"
    USER = "neo4j"
    PASSWORD = "gardens-board-admirals"

    export_neo4j_to_rdf(URI, USER, PASSWORD)
```